O comparație teoretică și experimentală a unor metode de rezolvare a satisfiabilității

4 martie 2026

Pintea Alexandra-Maria
Departamentul de Informatică
Facultatea de Matematică și Informatică
Universitatea de Vest, Timișoara, România
Email: alexandra.pintea06@e-uvt.ro

Rezumat

Această lucrare are ca obiectiv analiza și compararea a trei algoritmi esențiali utilizați pentru rezolvarea problemelor de satisfiabilitate în logica computațională: algoritmul de Rezoluție, algoritmul Davis–Putnam (DP) și algoritmul Davis–Putnam–Logemann–Loveland (DPLL). Studiul este realizat atât din punct de vedere teoretic, cât și din punct de vedere practic, întrucât am evidențiat modul de funcționare al fiecărui algoritm, eficiența lor în rezolvarea formulelor logice exprimate în formă normală conjunctivă (CNF), precum și aplicabilitatea lor în contexte reale. Compararea teoretică se concentrează pe mecanismele fundamentale ale algoritmilor și pe strategia de abordare a problemei de satisfiabilitate, precum și pe complexitatea timpului. În analiza practică am urmărit performanțele observate în testare, evidențiind limitările algoritmului de rezoluție simplă, îmbunătățirile aduse de algoritmul DP și superioritatea clară a algoritmului DPLL în majoritatea scenariilor moderne.

Cuvinte cheie: rezoluție, DP, DPLL, Davis-Putnam, Davis-Putnam-Logemann-Loveland, algoritm, satisfiabilitate, logica computațională, complexitate, eficiență, scalabilitate, formă normal conjunctivă.

Cuprins

Rezumat **2**

1 Introducere **2**
1.1 Declarație de originalitate 2
1.2 Instrucțiuni de citire 2

2 Descrierea algoritmilor de rezolvare a satisfiabilității **2**
2.1 Problema SAT 2
2.2 Algoritmul de rezoluție 3
2.3 Algoritmul Davis-Putnam 4
2.4 Algoritmul Davis-Putnam-Logemann-Loveland 4

3 Modelarea și implementarea problemei și soluției **4**
3.1 Manual 4
3.2 Implementarea algoritmilor 6

4 Experiment **9**
4.1 Comparație teoretică 9
4.1.1 Din punct de vedere al complexității timpului 9
4.1.2 Din punct de vedere al strategiei de rezolvare 9
4.2 Comparație experimentală 11

5 Comparație cu literatura **13**

6 Concluzii și direcții viitoare **14**
6.1 Concluzii 14
6.2 Direcții viitoare 14

Bibliografie **15**

1 Introducere

Un algoritm este o procedură de calcul bine definită care primește o mulțime de valori ca date de intrare și produce o mulțime de valori ca date de ieșire [3]. În mod normal, regula este apelată pentru a executa una sau mai multe operații. În această lucrare, scopul este de a compara trei algoritmi fundamentali utilizați în domeniul logicii și al inteligenței artificiale: Algoritmul de Rezoluție, Algoritmul Davis–Putnam (DP) și Algoritmul Davis–Putnam–Logemann–Loveland (DPLL). Vom vedea cum funcționează fiecare dintre ei, care sunt diferențele și în ce situații se descurcă mai bine sau mai puțin bine. Problema satisfiabilității apare în multe domenii, de la inteligență artificială până la verificarea automată a programelor. Deși există metode cunoscute care încearcă să o rezolve, nu toate sunt la fel de eficiente în orice situație. Din acest motiv, este interesant să vedem cum se comportă acești 3 algoritmi pe diferite tipuri de formule logice. În această lucrare, fiecare algoritm va fi explicat într-un mod simplu, pentru a înțelege ușor pașii prin care el ajunge la o concluzie. Apoi vom implementa acești algoritmi și îi vom testa pe câteva exemple, pentru a vedea cum se descurcă în practică. Lucrarea este împărțită în două părți principale: una teoretică, unde explicăm modul de funcționare și complexitatea algoritmilor, precum și una experimentală, unde îi comparăm în practică, pe formule în formă normală conjunctivă.

1.1 Declarație de originalitate

Declar pe proprie răspundere că această lucrare reprezintă rezultatul muncii personale, bazată pe cercetare proprie, toate sursele folosite fiind citate conform standardelor academice, iar implementările și testările aparținându-mi. Confirm că proiectul nu a fost prezentat anterior în cadrul altor activități academice, conferințe, concursuri sau lucrări similare și nu a fost reutilizat în alte scopuri decât cel actual. Prin această declarație îmi asum întreaga responsabilitate pentru conținutul lucrării și susțin cu sinceritate originalitatea și autenticitatea sa.

1.2 Instrucțiuni de citire

Secțiunea 2 definește formal problema SAT și ne oferă o bază de informații despre fiecare algoritm în parte. Secțiunea 3 prezintă un manual de utilizare și implementarea algoritmilor. Secțiunea 4 prezintă datele experimentale. Secțiunea 5 face o comparație cu lucrări relevante din literatură. Secțiunea 6 rezumă concluziile legate de realizarea acestei lucrări.

2 Descrierea algoritmilor de rezolvare a satisfiabilității

2.1 Problema SAT

Luăm în considerare formule propoziționale reprezentate prin mulțimi de clauze. O clauză este o mulțime de literali. Un literal este fie o formulă atomică, fie negația unei formule atomice. O formulă atomică și negația acesteia formează o pereche complementară. Considerăm o formulă atomică ca fiind o variabilă care poate fi interpretată ca având valoarea adevărat sau fals. Numărul de variabile dintr-o mulțime este dat de numărul de formule atomice distincte din acea mulțime. Un literal complementar al unei formule atomice este interpretat ca fiind opusul formulei atomice.

O clauză este interpretată ca o disjuncție a literalilor săi (aceasta este uneori reprezentată explicit ca o disjuncție și alteori ca o simplă mulțime de literali). O clauză vidă este interpretată ca fiind falsă. Se presupune că o clauză nu conține o pereche complementară. O mulțime de clauze este interpretată ca o conjuncție a clauzelor din acea mulțime. O mulțime vidă de clauze este interpretată ca fiind adevărată [7].
Mulțimea de clauze, cu operatorul de conjuncție între ele, formează o formulă booleană, în formă normal conjunctivă. Dată fiind o formulă booleană în forma normal conjunctivă (CNF-Conjunctive Normal Form), problema SAT presupune determinarea existenței unei atribuiri de valori de adevăr pentru variabile, astfel încât formula să fie adevărată.

Exemplu de satisfiabilitate

Presupunem formula în CNF:

$$(x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee \neg x_1) \wedge (\neg x_1 \vee \neg x_2)$$

Verificăm dacă există o atribuire a variabilelor care face formula adevărată. Alegem:

$$x_1 = \text{False}, \quad x_2 = \text{True}, \quad x_3 = \text{False}$$

Evaluarea clauzelor:

- $x_1 \vee \neg x_3 = \text{False} \vee \text{True} = \mathbf{True}$
- $x_2 \vee x_3 \vee \neg x_1 = \text{True} \vee \text{False} \vee \text{True} = \mathbf{True}$
- $\neg x_1 \vee \neg x_2 = \text{True} \vee \text{False} = \mathbf{True}$

Toate clauzele sunt adevărate $\Rightarrow$ formula este satisfiabilă. Dacă minim o clauză ar fi luat valoarea de adevăr fals $\Rightarrow$ formula era nesatisfiabilă.

2.2 Algoritmul de rezoluție

Algoritmul de rezoluție constituie o tehnică importantă pentru determinarea satisfiabilității, fiind folosită în domeniul Inteligenței Artificiale (AI) pentru a realiza raționamente logice. Scopul său principal este să determine dacă o anumită propoziție poate fi dedusă logic dintr-un set de informații deja cunoscute. Cu alte cuvinte, algoritmul ajută sistemul AI să stabilească dacă ceea ce vrem să demonstrăm se poate obține în mod logic pe baza a ceea ce știm deja. Algoritmul este aplicat atât în logica propozițională, cât și în logica predicatelor de ordinul întâi. De asemenea, se observă că calculul de rezoluție poate fi aplicat doar pe formule aflate în formă normală conjunctivă (CNF), adică pe mulțimi de clauze. În locul simbolului 0 (folosit pentru a desemna generic o formulă nesatisfiabilă), se obține clauza vidă.
Fie F o mulțime de clauze (o formulă în CNF). O *demonstrație prin rezoluție* (sau *refutare prin rezoluție*) este o secvență de clauze $\mathcal{R} = (C_1, C_2, \ldots, C_t)$ astfel încât ultima clauză este clauza vidă: $C_t =$. Mai mult, pentru fiecare $i = 1, 2, \ldots, t$, fie C_i este una dintre clauzele din F, fie este rezultatul aplicării regulii de rezoluție asupra a două clauze anterioare C_j și C_l, cu $j, l < i$ [4].

2.3 Algoritmul Davis-Putnam

Metoda Davis–Putnam este una dintre cele mai importante metode practice pentru problema satisfiabilității, SAT, în logica computațională. Algoritmul acesta poate fi considerat o îmbunătățire pentru rezoluție, fiind mai eficient comparativ cu rezoluția brută. Este un algoritm propus de către matematicianul Martin Davis și Hilary Putnam în 1960, algoritm care alege succesiv un literal pe care îl face adevărat, după care șterge toate clauzele care conțin acel literal, pentru că aceste clauze sunt deja adevărate și nu mai trebuie verificate în continuare. În plus, literalul negat este eliminat de peste tot din celelalte clauze, pentru că deja este fals și nu mai poate ajuta la satisfacerea vreunei clauze. Ideea de bază a algoritmului este următoarea: el alege, pas cu pas, un literal, adică o variabilă sau negația unei variabile și presupune că acesta este adevărat. Odată făcută această alegere, algoritmul face două operații importante pentru a simplifica formula: Se bazează pe propagarea unităților (adică rezoluția unitară și subsumarea unitară) și pe împărțirea în cazuri. Se știe că mulți factori afectează performanța metodei: structura de date folosită pentru clauze, alegerea variabilei pentru împărțire etc [6].

2.4 Algoritmul Davis-Putnam-Logemann-Loveland

În logică și știința calculatoarelor, algoritmul Davis–Putnam–Logemann–Loveland (DPLL) este un algoritm complet, fiind bazat pe căutare cu backtracking, adică întoarcere pe traseu, algoritmul încearcând să găsească soluții, iar atunci când vede că o anumită alegere duce la o situație imposibilă, întorcându-se (face "pași înapoi") și încercând alte variante, explorând astfel toate posibilitățile într-un mod organizat. Algoritmul este folosit pentru a decide satisfiabilitatea formulelor de logică propozițională scrise în formă normală conjunctivă. Practic, verifică dacă o formula logică poate fi adevărată, adică dacă există o combinație de valori adevărat-fals, astfel încât să satisfacă toate condițiile din acea formulă. Acest algoritm a fost introdus în anul 1961 de către 3 cercetători, Martin Davis, George Logemann și Donald W. Loveland și reprezintă o îmbunătățire a algoritmului anterior, mai vechi, Davis–Putnam, procedură bazată pe strategia de rezoluție, dezvoltată de Martin Davis și Hilary Putnam în 1960. DPLL a fost introdus ca o procedură de demonstrare pentru logica de ordinul întâi și are o istorie chiar mai îndelungată decât metoda Rezoluției [1]. În medie, DPLL este foarte rapid, întrucât cazurile în care o alegere greșită a literalului de ramificare duce la un timp de execuție exponențial sunt foarte rare. Există, totuși, formule pentru care fiecare strategie posibilă de selecție a literalului va duce inevitabil la un timp de execuție exponențial [2].

3 Modelarea și implementarea problemei și soluției

3.1 Manual

Această secțiune are un caracter tehnic și este destinată cititorilor interesați de detaliile de implementare, cum ar fi programatori, dezvoltatori, dar și utilizatori care vor să înțeleagă modul de

funcționare și testare al soluțiilor propuse pentru determinarea satisfiabilității formulelor logice. Problema abordată în cadrul acestei lucrări este problema de satisfiabilitate propozițională, SAT, modelată astfel încât să poată fi reprezentată pe calculator. Soluția presupune implementarea celor 3 algoritmi de satisfiabilitate în limbaj și testarea lor pe formule exprimate în forma normală conjunctivă, CNF. Urmează a experimenta diverse scenarii pentru a evalua cât de eficienți sunt acești algoritmi de satisfiabilitate. Timpul de execuție a fost realizat pe seturi de date variate. Instrucțiuni legate de implementarea și testarea algoritmilor:

- Manualul de sistem:

  În conceperea acestei lucrări, algoritmii de rezolvare a satisfiabilității precizați anterior au fost implementați utilizând limbajul de programare Python, pe același computer. Sistemul de operare folosit a fost Windows 10 Professional 64-bit, având un procesor Intel(R) Core(TM) i7-12650H cu frecvența de 2.30 GHz, 24 MB Cache și 16 GB memorie RAM de frecvență 3200 MHz.

- Manualul de utilizare:

  Pentru posibilitatea testării implementărilor algoritmilor de satisfiabilitate, aceștia au fost încărcați într-un repository public pe platforma Github. Link-urile corespunzătoare fiecărui algoritm se pot găsi mai jos, în secțiunea 3.2 a lucrării. Pentru rularea programelor este necesară rularea acestora pe un sistem compatibil cu limbajul Python. Acest lucru poate fi realizat fie local, fie direct în browser, prin intermediul unui compilator online. Accesând repository-ul Github menționat, se poate descărca sau copia codul sursă al fiecărui algoritm și se poate testa conform instrucțiunilor furnizate. Există mai multe opțiuni:

  - Fără instalare (recomandat pentru începători): Se accesează un site de tip compilator online, iar utilizatorul se asigură că limbajul de programare selectat este Python, preferabil versiunea 3, lipindu-se apoi codul algoritmului dorit în fereastra principală. Se apasă *"Run"* și se urmează instrucțiunile din consolă, întrucât se vor introduce clauze sub formă numerică, reprezentând un literal, respectiv negarea acestuia, în cazul numerelor negative. Rezultatul va apărea în partea de jos, în consolă. Pentru originalitate, în cazul algoritmului DPLL, am inclus în program testarea directă prin introducerea manuală a formulei dorite direct în codul sursă, pentru a-i verifica satisfiabilitatea, oferind o abordare diferită de testare față de introducerea clasică din consolă.
  - Pe calculator (opțional): Se poate instala Python de pe `https://www.python.org/downloads/`python.org, apoi se salvează fișierul cu extensia `.py` (exemplu: `rezolutie.py`) și se rulează din terminal cu comanda: `python [nume proiect].py`. Instrucțiunile de testare rămân aceleași.

Indiferent de algoritmul implementat sau de metoda aleasă pentru testare, după rulare, pe ecran se va afișa concluzia dacă formula este satisfiabilă sau nu. În plus, pentru a facilita înțelegerea modului în care sunt structurate formulele în formă normală conjunctivă, a fost implementat și un algoritm auxiliar, care se găsește în secțiunea 4.2, dedicat exclusiv generării clauzelor, fără a le evalua. Acest generator are rolul de a familiariza utilizatorii, în special pe cei mai puțin inițiați în logica propozițională, cu modul de scriere a clauzelor logice, ilustrând diferența dintre disjuncție, simbolizată prin $\vee$ și conjuncție, simbolizată prin $\wedge$. Astfel, se poate învăța cum se construiește o

formulă logică validă, înainte de a interacționa cu algoritmii compleți de satisfiabilitate. Această etapă reduce riscul introducerii unor formule greșite și contribuie la o mai bună înțelegere practică a modelului logic utilizat.

3.2 Implementarea algoritmilor

Algoritmii sunt implementați în Python, având în vedere performanța.

1. Algoritmul de rezoluție

- **Funcția** `pl_resolve(ci, cj)`**:**
  - Privește două clauze și caută literali opuși, de exemplu: x și $\neg x$.
  - Dacă găsește o astfel de pereche, construiește o clauză nouă fără acei literali.
  - Returnează noua clauză și un indicator boolean care arată dacă rezoluția a fost posibilă.
- **Funcția** `pl_resolution(cnf)`**:**
  - Parcurge toate perechile de clauze și aplică `pl_resolve`.
  - Dacă obține clauza vidă ($\emptyset$), rezultatul este nesatisfiabil.
  - Dacă nu se mai pot adăuga clauze noi, formula este considerată satisfiabilă.
- **Funcția** `read_clauses()`**:**
  - Permite utilizatorului să introducă clauze în format numeric.
  - Fiecare clauză se încheie cu `0`, marcând sfârșitul acesteia.
- **Blocul** `main`**:**
  - Apelează funcția de citire și pornește algoritmul de rezoluție.
  - Afișează dacă formula este satisfiabilă sau nu, în funcție de rezultat.

Codul sursă este disponibil la: `https://github.com/alexandraapintea/alexandra_pintea_SAT/blob/main/main%20-%202025-05-10T134813.475.py`.
Pentru măsurarea timpului de execuție, se adaugă comanda `import time` în antetul codului. În interiorul blocului principal `else` din main, înainte de apelul către funcția algoritmului, `pl_resolution(clauses)`, se inserează:

```
start = time.perf_counter()
result = pl_resolution(clauses)
end = time.perf_counter()
duration = (end - start) * 1000
```

În continuare, tot în interiorul acestui bloc principal de `else`, după `if result` și ultimul `else` din cadrul acestuia, se adaugă linia:

```
print(f"Timp de execuție: {duration:.4f} ms")
```

Aceasta este ultima linie de cod și va afișa durata execuției algoritmului în milisecunde, cu patru zecimale.

2. Algoritmul Davis–Putnam (DP)

- **Funcția** `dp(clauses)`**:**
  - Extrage toate atomii/variabilele din clauze.
  - Apelează recursiv funcția `dp_recursive`.
- **Funcția** `dp_recursive(atoms, clauses, values)`**:**
  - Dacă nu mai există clauze formula este satisfiabilă.
  - Dacă apare clauza vidă formula este nesatisfiabilă.
  - Caută și elimină literalul pur, adică cel care apare doar pozitiv sau doar negativ.
  - Aplică propagarea clauzelor unitare, cele cu un singur literal.
  - Dacă nu mai poate aplica nimic, încearcă valori `True`/`False` pentru literali neatribuiți, prin backtracking.
- **Funcția** `read_clauses()`**:**
  - Citește clauze în format numeric de la tastatură.
  - Verifică validitatea fiecărei clauze, adică dacă aceasta se termină cu 0.
- **Blocul** `main`**:**
  - Citește inputul și rulează algoritmul DP.
  - Afișează dacă formula este satisfiabilă sau nu.

Codul sursă este disponibil la: `https://github.com/alexandraapintea/alexandra_pintea_SAT/blob/main/main%20-%202025-05-10T134648.240.py`.
Pentru măsurarea timpului de execuție se procedează exact la fel cum s-a procedat pentru rezoluție.

3. Algoritmul Davis–Putnam–Logemann–Loveland (DPLL)

- **Clasa** `DPLLSolverOptimized`:
  - Primește o formulă în CNF sub formă de listă de liste.
  - Elimină duplicatele din fiecare clauză.
- **Metoda** `solve()`:
  - Inițiază procesul DPLL asupra formulei.
- **Metoda** `dpll(clauses)`:

  - Elimină literalii puri, adică cei care nu au complementul în formulă.
  - Aplică propagarea clauzelor unitare: dacă o clauză conține un singur literal, acel literal este adevărat.
  - Verifică dacă formula a fost satisfăcută; clauzele goale sugerează satisfiabilitatea.
  - Alege literalul cel mai frecvent și încearcă ambele valori, true sau false, recursiv, prin backtracking.

- **Metoda** `assign_literal`:
  - Atribuie un literal ca adevărat și simplifică formula.
  - Elimină clauzele care conțin literalul și modifică cele care conțin negația sa.

- **Metoda** `pure_literal_elimination`:
  - Caută literali care apar doar pozitiv sau doar negativ și îi elimină.

- **Metoda** `get_unit_literals`:
  - Returnează toți literalii din clauze unitare.

- **Metoda** `choose_literal`:
  - Alege literalul cel mai frecvent din întreaga formulă pentru ramificare.

Codul sursă este disponibil la: `https://github.com/alexandraapintea/alexandra_pintea_SAT/blob/main/main%20-%202025-05-10T134510.202.py`.
Pentru măsurarea timpului de execuție, se adaugă în antetul codului comanda `import time`, iar în main, după introducerea formulei, se rescriu următoarele comenzi:

```
solver = DPLLSolverOptimized(formula)
start_time = time.perf_counter()
result = solver.solve()
end_time = time.perf_counter()
duration_ms = (end_time - start_time) * 1000
print("Rezultat:", "Formula este SATISFIABILĂ" if result
else "Formula este NESATISFIABILĂ")
print(f"Timp de execuție: {duration_ms:.4f} ms")
```

4 Experiment

4.1 Comparație teoretică

4.1.1 Din punct de vedere al complexității timpului

Din punct de vedere al complexității timpului, vom compara algoritmii de satisfiabilitate logică după cel mai rapid timp de execuție, timpul mediu și cel mai lent timp de execuție.

Algoritm SAT	**Caz favorabil**	**Timp mediu estimat**	**Caz nefavorabil**
Rezoluție	$O(n)$	$O(2^n)$	$O(2^n)$
Davis–Putnam (DP)	$O(n)$	$O(2^n)$	$O(2^n)$
DPLL	$O(n)$	$O(c^n)$, unde $c < 2$	$O(2^n)$

Tabela 1: Comparația complexității între algoritmii SAT: Rezoluție, DP și DPLL

Pe baza tabelului de mai sus, pot fi observate următoarele aspecte:

- Caz favorabilul este identic, întrucât toți cei 3 algoritmi au aceeași complexitate: O(n), dat fiind cazul în care instanța este foarte simplă și conține clauze unitare sau poate fi rezolvată imediat, astfel că toți algoritmii se comportă eficient.
- Complexitatea cazului nefavorabil este aceeași pentru toți cei 3 algoritmi, ajungând, în cel mai rău caz, la complexitatea exponențială O(2^n).

4.1.2 Din punct de vedere al strategiei de rezolvare

Cele 3 metode analizate, rezoluția, DP și DPLL folosesc abordări diferite pentru determinarea satisfiabilității formulelor logice propoziționale exprimate în formă normală conjunctivă. Mai jos se poate consulta un tabel care explică concis modul de abordare și determinare a satisfiabilității în cadrul fiecăruia dintre acești 3 algoritmi.

Tabela 2: Metoda de rezolvare integrată a fiecărui algoritm SAT

Algoritm SAT	**Metodă de rezolvare detaliată**
Rezoluție	• Se caută două clauze care conțin literali complementari, de exemplu A și ¬A. • Se aplică regula de rezoluție: se elimină literalul complementar și se formează o clauză nouă. • Clauza nouă se adaugă la mulțimea de clauze existente. • Se repetă procesul până când: – se obține clauza vidă **nesatisfiabil**, sau – nu se mai pot face rezoluții **posibil satisfiabil**.
DP	• Se selectează o variabilă din formulă. • Se caută toate clauzele ce conțin variabila și negația ei. • Se aplică rezoluția completă între toate combinațiile posibile. • Se elimină clauzele inițiale și se adaugă cele rezultate. • Procesul se repetă până când: – se obține clauza vidă **nesatisfiabil**, sau – nu mai sunt variabile **satisfiabil**.

Algoritm SAT	Metodă de rezolvare detaliată
DPLL	• Se alege un literal și se presupune o valoare, true/false. • Se aplică propagarea clauzelor unitare. • Se elimină clauzele satisfăcute și se reduc cele afectate. • Dacă apare contradicție, adică clauză vidă, se face backtracking. • Dacă toate clauzele sunt satisfăcute **SAT**, altfel **UNSAT**.

4.2 Comparație experimentală

Pentru familiarizarea cu satisfiabilitatea și, în special, cu forma standardizată a clauzelor, înainte de comparația propriu-zisă a celor 3 algoritmi implementați, vom implementa un algoritm care generează clauze în forma normal conjunctivă, formă cerută pentru a putea utiliza acești algoritmi. Algoritmul de generare a clauzelor este disponibil la: `https://github.com/alexandraapintea/alexandra_pintea_SAT/blob/main/main%20-%202025-05-10T141111.241.py`. Utilizatorul introduce de la tastatură clauze în format numeric, fiecare clauză fiind reprezentată de o listă de numere întregi, care corespund unor variabile logice. Fiecare număr pozitiv reprezintă o variabilă (1-A, 2-B etc), iar fiecare număr negativ reprezintă negația acesteia (1-not A, 2-not B etc). Fiecare clauză se încheie obligatoriu cu 0, care indică închiderea clauzei curente și un semnal pentru a trece la următoarea. După introducerea clauzelor în format numeric, pe ecran acestea vor fi afișate în formă normal conjunctivă, sub formă de conjuncție de literali. După acest pas important preliminar, este timpul pentru a trece propriu-zis la studiul de caz, precum concentrarea pe implementarea celor 3 algoritmi de determinare a satisfiabilității, testarea acestora și remarcarea diferențelor dintre ei în diferite cazuri. Algoritmii au fost testați pe același computer, pe un număr generos de formule CNF generate aleatoriu, pentru a observa comportamentul general al fiecărei metode în contexte variate. Pentru claritate și concizie, în cadrul experimentului prezentat în lucrare au fost selectate 7 cazuri relevante, reprezentând formule de complexitate diferită. Acestea au fost utilizate manual ca date de test pentru compararea timpului de execuție, în milisecunde al algoritmilor Rezoluție, DP și DPLL. Pentru fiecare algoritm, la rularea programului, s-a avut în vedere primul timp de execuție obținut, fără reluarea sau medierea rezultatelor, iar acest timp a fost utilizat drept punct de referință în tabelul comparativ, rezultatele fiind sintetizate în tabelele de mai jos.
S-au considerat următoarele formule CNF ca exemple de intrare:

- *Cazul 0:* $x_1 \wedge \neg x_1$
- *Cazul 1:* $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$

- *Cazul 2:* $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$
- *Cazul 3:* $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_3)$
- *Cazul 4:* $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_4) \wedge (\neg x_2 \vee \neg x_4) \wedge (x_5) \wedge (\neg x_3 \vee \neg x_5) \wedge (x_6) \wedge (\neg x_6 \vee x_7) \wedge (\neg x_7 \vee \neg x_8) \wedge (x_8 \vee \neg x_1)$
- *Cazul 5:* $(x_1 \vee x_2) \wedge (x_3) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_4) \wedge (x_4 \vee x_5) \wedge (\neg x_5 \vee x_6) \wedge (\neg x_6) \wedge (\neg x_1 \vee \neg x_2 \vee x_6)$
- *Cazul 6:* $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_4) \wedge (\neg x_2 \vee x_5) \wedge (\neg x_3 \vee x_6) \wedge (\neg x_4 \vee \neg x_5) \wedge (\neg x_5 \vee \neg x_6) \wedge (x_7) \wedge (\neg x_7 \vee x_1) \wedge (\neg x_7 \vee \neg x_2)$

Caz	**Nr. variabile**	**Satisfiabilă?**	**DPLL (ms)**	**DP (ms)**	**Rezoluție (ms)**
Cazul 0	1	*Nu*	0.0107	0.0274	0.0330
Cazul 1	3	*Da*	0.0094	0.0490	0.0610
Cazul 2	3	*Da*	0.0224	0.0594	0.2334
Cazul 3	3	*Da*	0.0277	0.0528	5.4659

Tabela 3: Comparație timpi de execuție a algoritmilor DPLL, DP și Rezoluție pe formule CNF

Caz	**Nr. variabile**	**Satisfiabilă?**	**DPLL (ms)**	**DP (ms)**	**Rezoluție (ms)**
Cazul 4	8	*Da*	0.0300	0.1277	9.2795
Cazul 5	6	*Nu*	0.0412	0.0824	0.6757
Cazul 6	7	*Da*	0.0324	0.0948	418.6930

Tabela 4: Comparație timpi de execuție pentru formule CNF mai mari

Pe baza acestor 2 tabele de comparație realizate, se pot reuni câteva concluzii privind eficiența și performanța algoritmilor DP, DPLL și rezoluție, în rezolvarea formulelor CNF de dimensiuni variate.

- La formule mici, în Tabela 3, toate cele 3 metode reușesc să rezolve formule într-un timp rezonabil, iar DPLL și DP sunt foarte apropiate ca performanță.
- La formule mari, în Tabela 4, intervin diferențe semnificative între performanța algoritmilor:
  - DPLL rămâne constant sub 0.1 ms, chiar și la multe variabile.
  - DP are o scalare puțin mai slabă, dar reușește să rămână sub 0.2 ms.
  - Rezoluția înregistrează o creștere exponențială a timpului de execuție odată cu mărirea complexității formulei, atingând valori considerabile, de până la 418 ms.

Din punct de vedere experimental, cei 3 algoritmi prezintă diferențe semnificative în ceea ce privește eficiența. Deși testele au fost realizate pe un eșantion restrâns de formule CNF, acestea au fost suficiente pentru a observa deosebirile de performanță. Astfel, în cazul formulelor mici, cei 3 algoritmi de satisfiabilitate oferă timpi de execuție similari, cu o ușoară superioritate a algoritmului DPLL în unele cazuri. Pe măsură ce formulele devin mai mari, diferențele sunt tot mai pronunțate, DPLL reușind să își păstreze eficiența și să ia un avantaj detașat, DP înregistrând o creștere ușoară, moderată a timpului de execuție, iar rezoluția, în unele scenarii, explozând în timpi de execuție, generând un număr mare de clauze noi. În circumstanțe mai complexe și moderne, DPLL rămâne cel mai eficient algoritm pentru determinarea satisfiabilității unei formule în CNF, urmat de DP, respectiv, la final, rezoluția.

5 Comparație cu literatura

Pentru verificarea și compararea rezultatelor obținute în această lucrare, vom analiza diferența, dacă aceasta există, de rezultate cu anumite implementări din literatura de specialitate. Spre exemplu, putem avea ca punct de referință algoritmi publicați în articole științifice destinate, precum cel propus de Hantao Zhang și Mark Stickel pentru Davis–Putnam [5].

Se remarcă diferențe majore între implementarea proprie și implementarea academică prezentată în literatură, întrucât algoritmul implementat în lucrarea mea urmează o abordare recursivă simplă, în care se aplică propagarea unităților, se caută literali puri și și se realizează ramificarea pe baza primului literal disponibil, fiind conceput pentru ușurința în testare și înțelegere. Algoritmul lui Zhang este mult mai rafinat și optimizat, urmând o structură care include subsumarea clauzelor, propagarea rapidă, bazată pe indexare eficientă, "trie", fiind susținut de măsurători experimentale riguroase pe multiple instanțe SAT, demonstrând performanțe semnificativ mai bune în termeni de timp de execuție și număr de ramificări. Spre deosebire de algoritmul academic, care este prezentat într-un format destul de teoretic și necesită adaptări pentru a putea fi implementat practic, respectiv testat, versiunea dezvoltată în această lucrare oferă un cod clar și accesibil, care permite testarea directă prin introducerea interactivă a clauzelor CNF din consolă, putând fi utilizat chiar și de cei fără experiență avansată în domeniu.

- Lucrarea conține:
  - posibilitatea introducerii interactive a clauzelor CNF în format numeric, direct din consolă.
  - o implementare practică, testabilă fără a necesita conversii sau adaptări suplimentare.
- Lucrarea nu conține:
  - utilizarea unor structuri avansate de date, spre exemplu: trie, pentru optimizarea propagării unităților.
  - optimizări avansate precum subsumpția clauzelor sau alegerea literalului pe baza euristicilor sofisticate.

6 Concluzii și direcții viitoare

6.1 Concluzii

Așadar, studiul efectuat reprezintă o comparație între 3 algoritmi de satisfiabilitate logică: Algoritmul de Rezoluție, Algoritmul Davis-Putnam (DP) și Algoritmul Davis-Putnam-Logemann-Loveland (DPLL). Prin implementarea, analiza și compararea acestor algoritmi, am vrut să vedem cum funcționează fiecare dintre ei atât din punct de vedere teoretic, cât și practic și în ce situații sunt mai potrivite. În primul rând, s-a realizat analiza teoretică, în care s-a comparat complexitatea algoritmică, precum și ideea de rezolvare. În al doilea rând, s-au efectuat teste practice pe formule logice în formă normală conjunctivă, CNF, variind atât numărul de variabile, cât și structura clauzelor. Ce am observat este că algoritmul de rezoluție, deși simplu și corect teoretic, devine rapid ineficient pe formule mai mari. Algoritmul DP aduce o oarecare îmbunătățire, dar poate genera multe clauze suplimentare, crescând dimensiunea formulei. DPLL, în schimb, s-a dovedit cel mai eficient în majoritatea cazurilor, fiind mai rapid și mai scalabil. În procesul practic, am întâlnit câteva dificultăți, cum ar fi dorința de generare a clauzelor pe baza reprezentării numerice, pentru originalitate, precum și testarea multiplă a algoritmilor pe o varietate de formule, care a impus o atenție sporită, pentru verificarea corectitudinii implementării. În această lucrare, scopul nu a fost să măsurăm neapărat care algoritm este cel mai rapid, ci mai degrabă a înțelege cum funcționează fiecare și ce îl face diferit față de ceilalți, cât de bine se descurcă în anumite cazuri și unde încep să dea semne de slăbiciune. După un număr considerabil de comparații între algoritmi, am ajuns la concluzia că nu există un algoritm care să ofere soluții optime pentru orice problemă SAT, deoarece fiecare algoritm are punctele sale forte și propriile limitări. Prin urmare, consider că cea mai "bună metodă de sortare" sau "cel bun algoritm" este îmbinarea utilizării punctelor forte ale fiecăruia. Astfel, asocierea acelor caracteristici de la cei 3 algoritmi de rezolvare a satisfiabilității, într-un mod inteligent, ar putea reprezenta o direcție promițătoare în obținerea rezultatelor cât mai optime dorite.

6.2 Direcții viitoare

Direcțiile viitoare pentru această lucrare vizează dorința extinderii și aprofundării analizei realizate asupra algoritmilor de Rezoluție, DP și DPLL. O posibilă continuare ar putea fi investigarea și implementarea unor variante optimizate ale algoritmului DPLL, cum ar fi integrarea tehnicilor de învățare a conflictelor, Conflict-Driven Clause Learning – CDCL, pentru o mai bună eficiență practică, fiind un algoritm mai modern. De asemenea, în extinderea lucrării doresc realizarea unor grafice comparative care să evidențieze performanța fiecărui algoritm în funcție de numărul de variabile și de complexitatea formulelor logice testate, precum și o analiză a punctelor slabe, limitărilor, respectiv a punctelor forte fiecărui algoritm. Experimentul ar putea fi extins și prin rularea testelor pe o gamă mai variată de sisteme de procesoare. Consider că aceste direcții viitoare urmăresc îmbunătățirea atât a corectitudinii și complexității analizei făcute asupra algoritmilor, cât și a părții practice, legate de comparația algoritmilor prin diverse testări în profunzime, prin aplicarea acestora în scenarii reale și variate.

Referințe

[1] Baumgartner, Peter et al., "A first-order logic Davis-Putnam-Logemann-Loveland procedure", în *AI in the new Millenium, Morgan Kaufmann, Seattle* (2002).

[2] Biere, Armin, Heule, Marijn și Maaren, Hans van, *Handbook of satisfiability*, vol. 185, IOS press, 2009.

[3] Carabineanu, Adrian, *Structuri de date*, Matrix Rom, 2006.

[4] Schöning, Uwe și Torán, Jacobo, *The Satisfiability Problem: Algorithms and Analyses*, vol. 3, Lehmanns media, 2013.

[5] Zhang, Hantao și Stickel, Mark, "Implementing the davis–putnam method", în *Journal of Automated Reasoning* 24.1 (2000), pp. 277–296.

[6] Zhang, Hantao și Stickel, Mark E, *Implementing the Davis-Putnam algorithm by tries*, Citeseer, 1994.

[7] Zhang, Wenhui, "Number of models and satisfiability of sets of clauses", în *Theoretical Computer Science* 155.1 (1996), pp. 277–288.